

◆ PHASED DELIVERY PLAN

KINETIC TYPE

Implementation Plan

Six phases from repository setup to public launch — expanding the original three-phase engineering roadmap into a full delivery plan with milestones, owners, and exit criteria.

VERSION

1.0

STATUS

Draft for Review

DATE

31 Aug 2026

HORIZON

~17 Weeks to GA

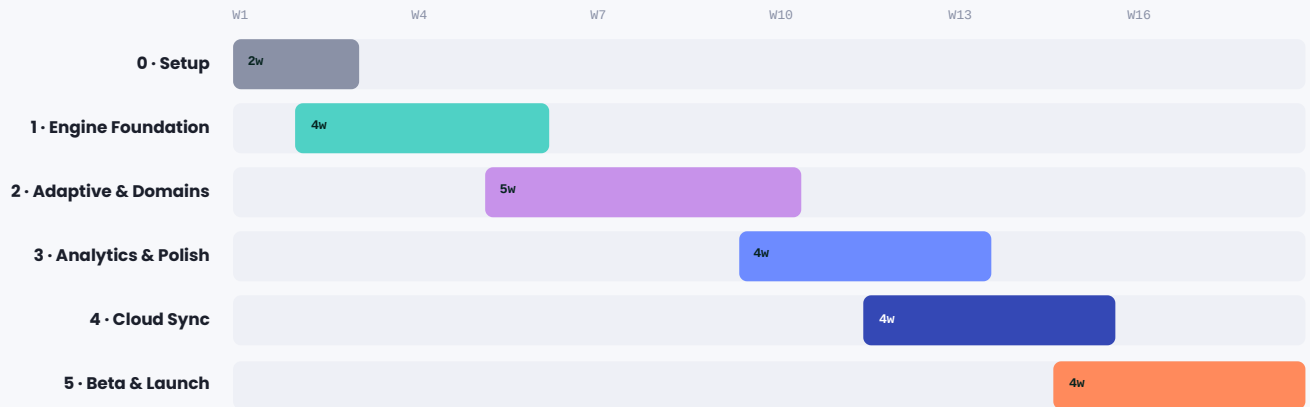
Confidential & Proprietary

Table of Contents

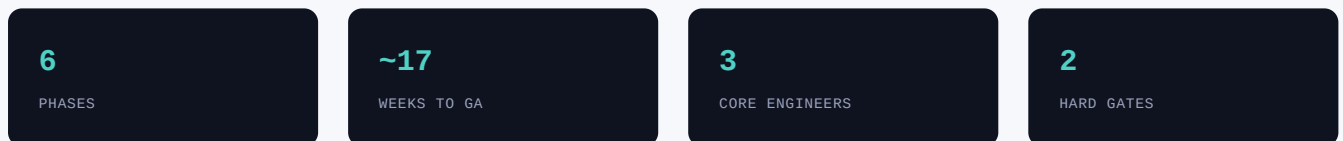
01	Timeline Overview	03	02	Phase 0 — Setup & Foundations	04
03	Phase 1 — Engine Foundation	05	04	Phase 2 — Adaptive Engine & Domains	06
05	Phase 3 — Visual Analytics & Polish	07	06	Phase 4 — Cloud Sync & Accounts	08
07	Phase 5 — Beta & Launch	09	08	Team & Skills	10
09	Risk Register by Phase	10			

01 Timeline Overview

The source architecture spec defined three engineering phases (Engine Foundation, Adaptive Engine & Domains, Visual Analytics & Polish). This plan wraps them with a setup phase and adds cloud/launch phases required to actually ship a product, not just an engine.



Bars overlap intentionally — Phase 4 (Cloud Sync) begins while Phase 3 (Analytics) is finishing, since they touch largely disjoint parts of the codebase. Total horizon: ~17 weeks to General Availability.



02 Phase 0 — Setup & Foundations

2 WEEKS

Nothing user-facing ships in this phase — it exists so Phase 1 starts on solid ground.

TASK	DETAIL
Repository & tooling	Vite + React + TypeScript scaffold, ESLint/Prettier, Vitest, Playwright config
Firebase project provisioning	Create dev/staging/prod projects; configure Auth, Firestore, Hosting, Emulator Suite
CI/CD pipeline	GitHub Actions: lint → test → build → Hosting preview channel per PR
Design tokens import	Color, type scale, and spacing tokens from the UI/UX Brief encoded as Tailwind config / CSS variables
Architecture spike	Prove out the Web Worker + <code>postMessage</code> + Canvas rendering pipeline end-to-end with dummy data

EXIT CRITERIA

A "hello world" keystroke can be captured on the main thread, timestamped, sent to a worker, and echoed back to a canvas paint — proving the core latency-critical pipeline before any real feature work begins.

03 Phase 1 — Engine Foundation

4 WEEKS

Maps directly to the original spec's Phase 1. Goal: a functioning, honest typing test with zero adaptivity yet — but with rendering and timing precision already at production quality.

WORKSTREAM	DELIVERABLES
High-precision event system	Sub-key event listener capturing keydown/keyup via <code>performance.now()</code> ; visibility-change sample discarding
Virtualized renderer	Canvas-based character stream renderer; block caret; no native inputs
Core metrics	Raw WPM, Net WPM, and per-keystroke latency calculations, unit tested against fixed fixtures
Local persistence skeleton	Dexie.js schema for <code>ngram_stats</code> and <code>session_logs</code> stood up (Backend Schema §02)
Static "Prose" domain	One working domain lexer to validate the full pipeline before multiplying to three

EXIT CRITERIA

A user can complete a full Prose-mode session and see accurate WPM/accuracy — with input-to-paint latency already measured under the 5ms p95 budget.

04

Phase 2 — Adaptive Engine & Domains

5 WEEKS

The heart of the product. Maps to the spec's Phase 2, expanded with the spaced-repetition layer implied by the original data schema.

WORKSTREAM	DELIVERABLES
N-gram latency matrix	EWMA calculation engine running in the Web Worker; persisted & loaded from IndexedDB
Weighting algorithm	$W(P) = L(P) \times (1 + \alpha \cdot E(P))$ implemented, tunable α , unit-tested against synthetic weak-pair fixtures
Dynamic Material Synthesizer	Text Selection Protocol (TRD §04) implemented — 70/30 targeted/flow assembly
Dev & CLI lexers	Two additional domain corpora: nested code blocks/operators; shell flags/paths/regex
Spaced-repetition engine	<code>srs_queue</code> scheduling logic reintroducing mastered pairs at decaying frequency

EXIT CRITERIA (ADAPTIVE LOOP VERIFIED)

A synthetic test harness injects a deliberately slow key-pair; within one session, that pair's frequency in the generated text stream measurably increases — the loop's core promise, proven automatically in CI.

05

Phase 3 — Visual Analytics & Polish

4 WEEKS

Maps to the spec's Phase 3. This is where the product becomes legible and trustworthy to the user, per the UI/UX Brief.

WORKSTREAM	DELIVERABLES
Latency heatmap	Keyboard-shaped visualization component (UI/UX Brief §08), colorblind-safe ramp
Micro-pause & fatigue graphs	Anomaly detection surfaced as an annotated timeline on the results screen
History & trends	Rolling charts of flagged-pair improvement over time
Motion & interaction pass	Full implementation of the Motion table (UI/UX Brief §09), incl. reduced-motion variants
Accessibility audit	WCAG 2.1 AA contrast pass, keyboard operability of all chrome, focus ring implementation

EXIT CRITERIA

Independent accessibility audit passes; a first-time user in a moderated usability test can correctly explain what the heatmap is telling them within 15 seconds of seeing it.

06 Phase 4 — Cloud Sync & Accounts

4 WEEKS

New relative to the original spec, required because the product needs to survive a cleared browser and follow a user across devices. Runs partially in parallel with Phase 3.

WORKSTREAM	DELIVERABLES
Firebase Auth integration	Anonymous sign-in on first load; Email/Google linking flow (App Flow §06)
Firestore mirror	Collections per Backend Schema §03; security rules written & emulator-tested
Sync engine	Write-through outbox queue, debounced batch flush, occurrence-weighted merge (Backend Schema §07)
Cloud Functions	Nightly aggregation job; getCorpusManifest callable; account-deletion purge function
App Check & rate limiting	Abuse prevention on all callable/HTTPS endpoints

EXIT CRITERIA

A simulated two-device test — writing conflicting `ngram_stats` updates on both while offline, then reconnecting both — resolves with no data loss and a mathematically correct merged average.

07

Phase 5 — Beta & Launch

4 WEEKS

WORKSTREAM	DELIVERABLES
Closed beta	50–100 users across the three target personas; weekly telemetry review against PRD KPIs
Performance hardening	Real-user monitoring via Firebase Performance; address any p95 latency regressions found under real load
Cross-browser QA	Full pass across Chrome/Edge/Firefox/Safari on the target device matrix
Content & corpus review	Legal/licensing check on all Dev/CLI/Prose source material
Launch readiness review	Final checklist against PRD §11 Release Criteria; go/no-go decision

HARD GATE

GA does not ship until the D7 retention and 14-day latency-improvement KPIs (PRD §04) are trending toward target in beta data — this is a data-gated launch, not a date-gated one.

08 Team & Skills

ROLE	FOCUS	PHASES MOST ENGAGED
Frontend/Performance Engineer	Canvas renderer, event bus, worker architecture	0, 1, 3
Full-Stack Engineer	Adaptive algorithm, Firebase integration, sync logic	2, 4
Product Designer	UI/UX Brief execution, motion, accessibility	3, 5
Product Manager	KPI tracking, beta program, launch gate	0, 5
QA / Test Engineer (part-time)	Cross-browser matrix, sync conflict testing	3, 4, 5

09 Risk Register by Phase

PHASE	RISK	MITIGATION
1	Canvas renderer latency targets prove unreachable on low-end hardware	Establish device performance floor early via the Phase 0 spike; degrade gracefully to a simpler renderer if needed
2	Adaptive algorithm feels manipulative/unnatural to real users	Usability testing at end of Phase 2 before investing further in polish
4	Sync merge logic has an undiscovered edge case causing data loss	Property-based/fuzz testing of the merge function; staged rollout with a kill-switch on sync
5	Beta KPIs miss target, forcing a scope or timeline change	Weekly KPI review from week 1 of beta, not just at the end, to allow course-correction

End of Implementation Plan — Kinetic Type v1.0. Confidential & Proprietary.